Digital circuit design (RT-level)

M. Sonza Reorda

Politecnico di Torino
Dip. di Automatica e Informatica



Outline

- **Introduction**
- **Gate-level design**
- **RT-level design**

RT-level design

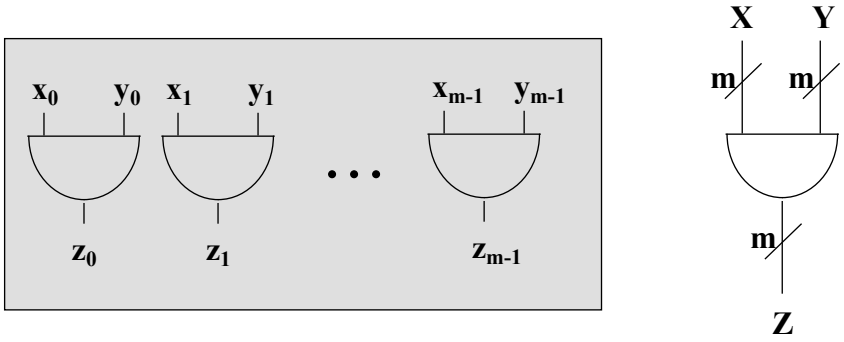
Features:

- **the main data unit is the *word***
- **the components used in the project are blocks (combinational or sequential) for storing or transforming words.**

Components

- **Combinational components**
  - **Logic gates with word parallelism**
  - **Multiplexers**
  - **Decoders and encoders**
  - **Arithmetic units (ALUs, adders, etc.)**
- **Sequential components**
  - **Registers**
  - **Counters**
  - **Busses**
  - **Memories.**

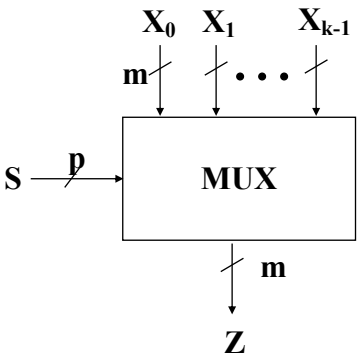
Logic gates with word parallelism



5

Multiplexer

A multiplexer (MUX) connects one out of *k* data sources X_i to a data destination Z, depending on the value of *p* selection signals S, where $k=2^p$.



6

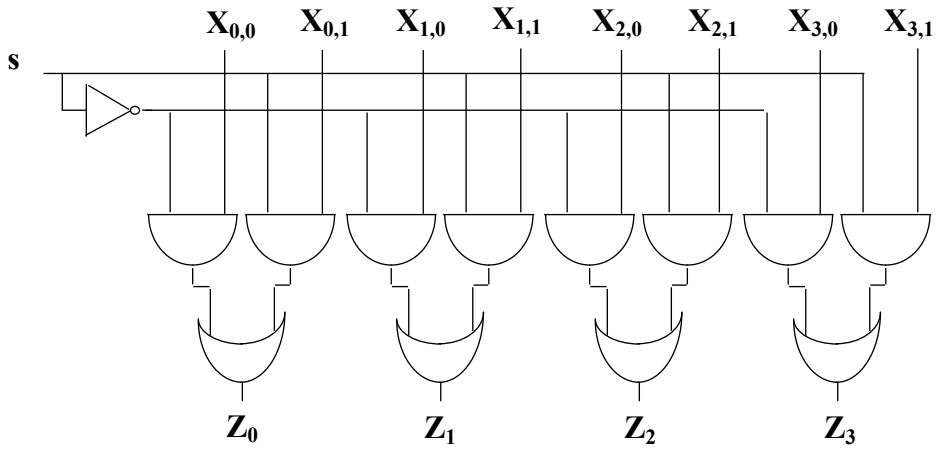
Multiplexer: behavior

The truth table of a 2 → 1 MUX with data inputs X_0 and X_1, selection input S and output Z is the following:

S	Z
0	X_0
1	X_1

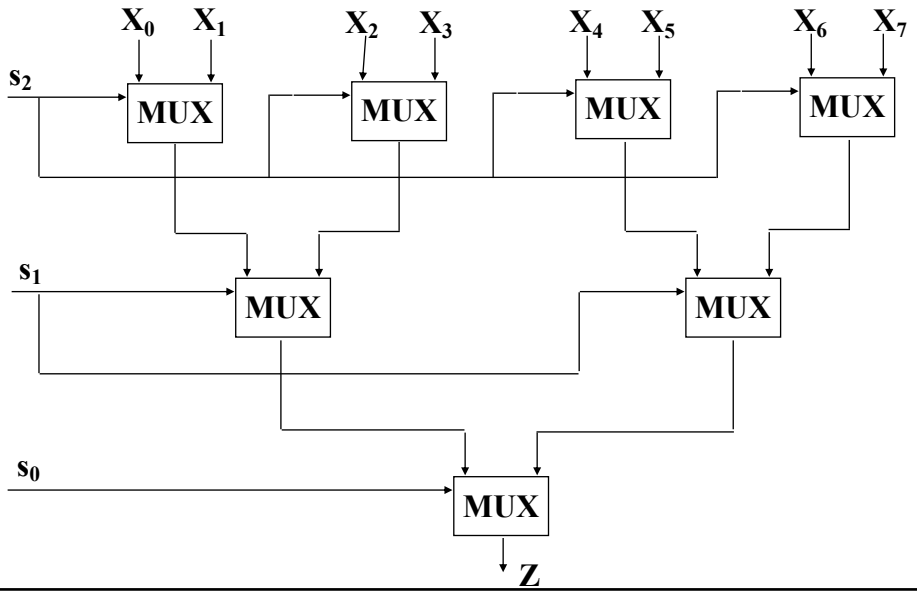
7

Multiplexer: implementation



8

Multiplexer: cascade connection

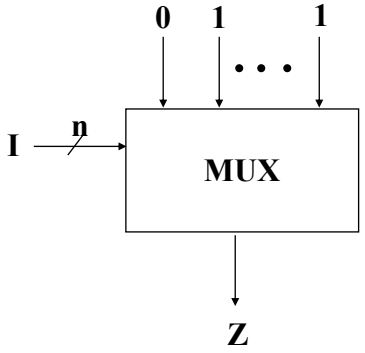


9

Multiplexer: usage in synthesis

MUXs can be used to implement a generic combinational function.

$Z=f(i_0,i_1,...,i_{n-1})$



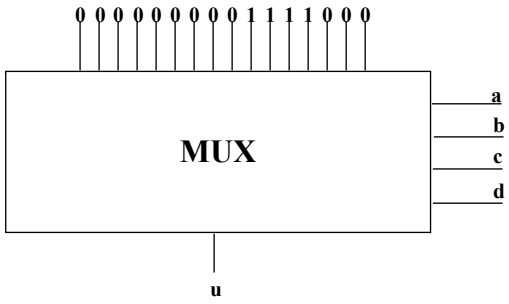
Each data input corresponds to a minterm.

10

Synthesis via MUXs: example

We want to implement a circuit with 4 inputs a, b, c, d whose output u holds the value 1 when 8<(abcd)<13.

A possible implementation is



11

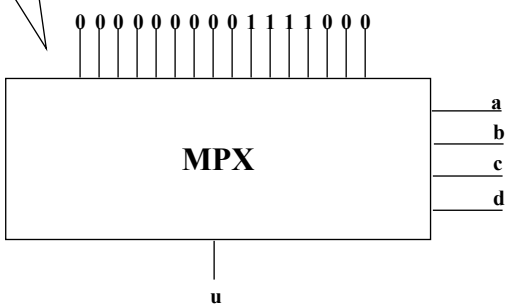
Synthesis via MUXs: example

This MUX can be implemented with a tree of 15 2 → 1 MUXs.

The total cost in terms of logic gates is then 15 × 4 = 60 logic gates.

We want to implement a circuit with 4 inputs a, b, c, d whose output u holds the value 1 when 8<(abcd)<13.

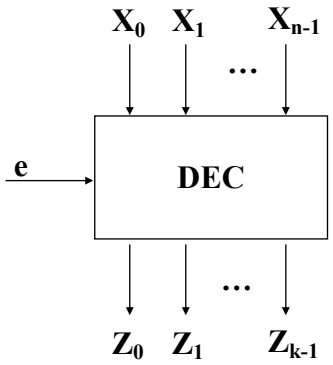
A possible implementation is



12

Decoder

A decoder (DEC) has *n* input signals and 2^n output signals; out of the these, only the one whose index corresponds to the input value is active.



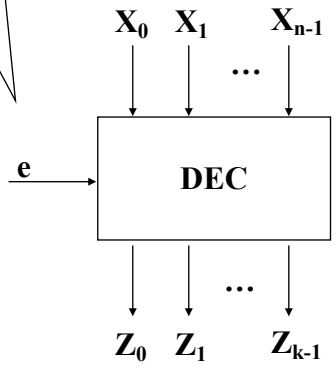
k=2^n

13

Decoder

In some cases there is an *enable* signal; when it is not active no output is active (whatever the value of the other inputs).

owns *n* input signals and 2^n output signals; only the one whose index corresponds to the input v… active.



k=2^n

14

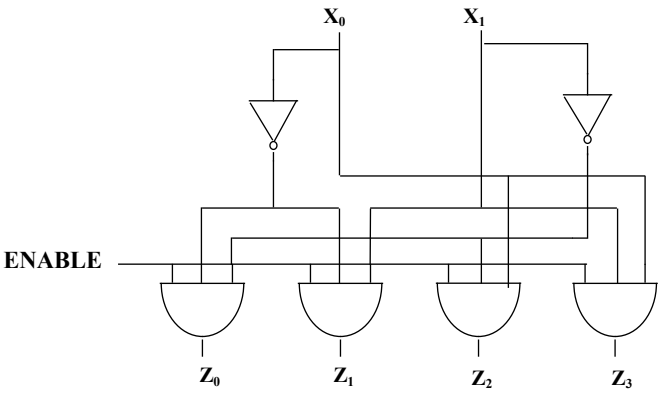
Decoder: behavior

The truth table of a 2→4 decoder is the following:

X	e	Z
00	1	0001
01	1	0010
10	1	0100
11	1	1000
-	0	0000

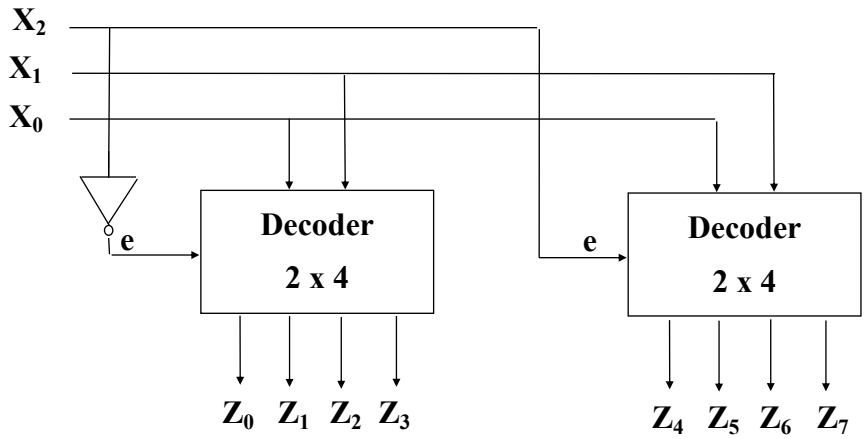
15

Decoder: implementation



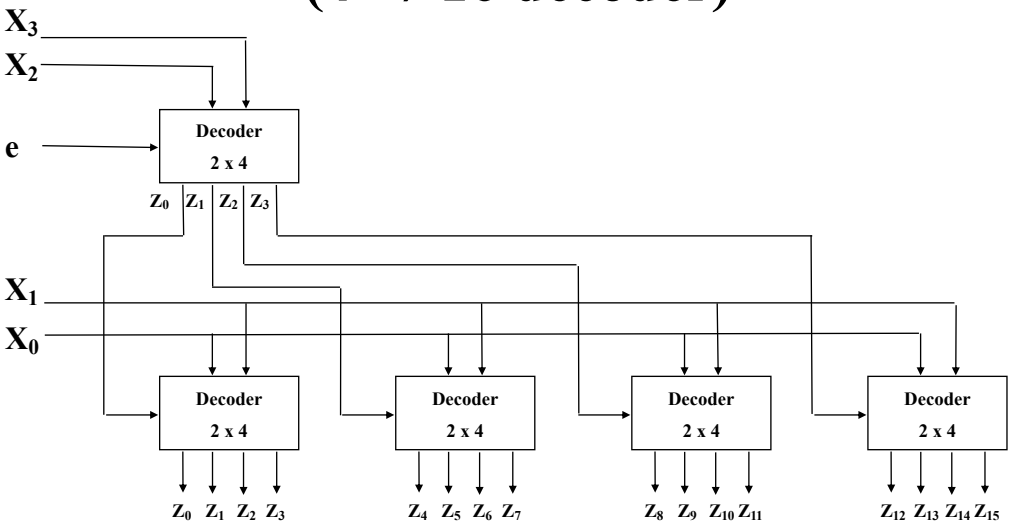
16

Decoders: cascade connection (3 → 8 decoder)



17

Decoders: cascade connection (4 → 16 decoder)

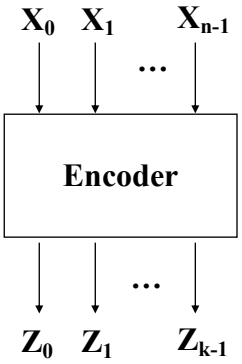


18

Encoder

An Encoder has 2^k input lines and k output lines.

The value on the output lines corresponds to the index of the active input line.



$n=2^k$

19

Encoders: behavior

The truth table of a 4→2 Encoder is the following:

X	Z
0001	00
0010	01
0100	10
1000	11

20

Priority Encoders

If more than one input line is active at a time the result may be undefined.

In other cases, the outputs show the index of the active input line with higher priority (*priority encoder*).

21

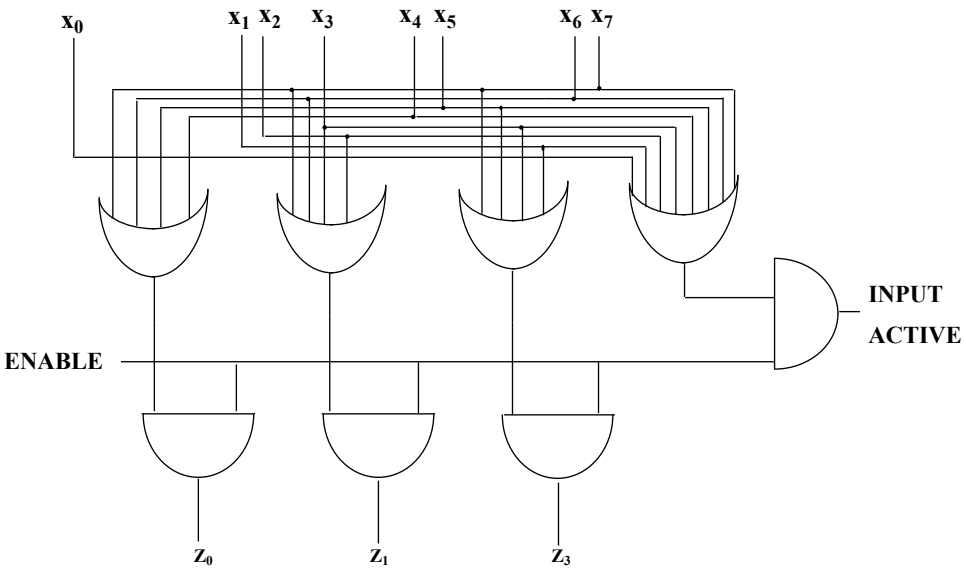
Priority Encoders: behavior

The truth table of a 4→2 Priority Encoder is the following:

X	Z
0000	00
0001	00
0010	01
0011	01
0100	10
0101	10
0110	10
0111	10
1 - - -	11

22

Encoder 8→3 with enable



23

Arithmetic modules

They may have variable complexity depending on

- supported data type (integer, full with sign, decimal)
- type of supported operations (sum, subtraction, multiplication, division, trigonometric operations)
- speed (combinational or sequential solutions).

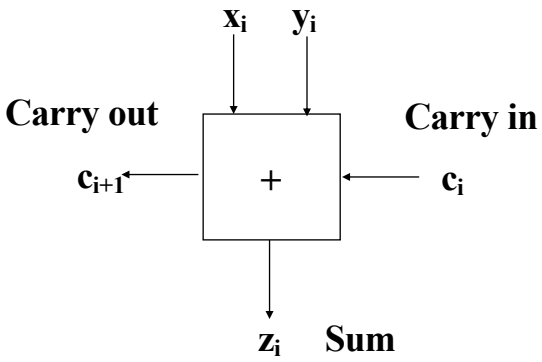
24

Full-Adder

It computes the sum of 2 bits x_i, y_i and an input carry bit c_i, producing an output data bit z_i and an output carry c_{i+1}:

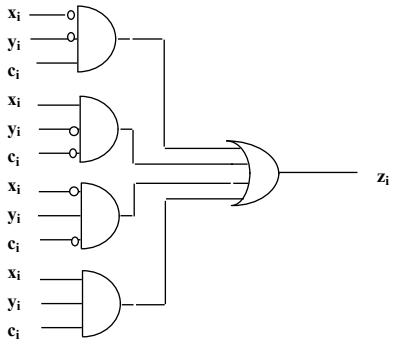
$$z_i = x_i \oplus y_i \oplus c_i$$
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

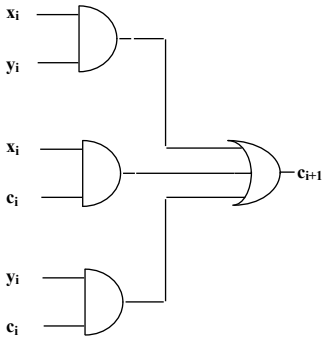
A	B	Carry -In	Sum	Carry -Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



25

Full-Adder: possible implementation





26

Adders

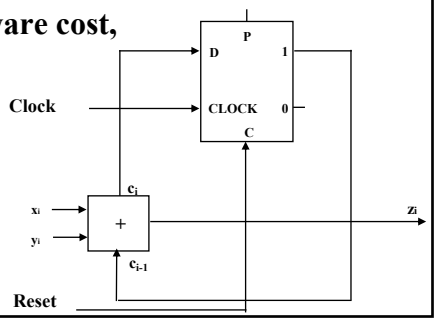
They can be implemented following three alternative solutions:

- serial adders
- combinational adders
- modular combinational adders.

27

Serial adder

- The 2 operands are added bit by bit starting from the Least Significant Bit
- The Flip Flop stores the Carry bit produced at each step and provides it to the following step
- At first the FF must be cleared
- This solution
  - requires the minimum hardware cost, which is independent on n (the operand size)
  - it requires the maximum response time (equal to n clock periods).



28

Combinational adder

The best solution in terms of speed corresponds to a 2-level circuit designed to add 2 numbers on n bits and produce n + 1 output bits.

Such a circuit can be synthesized from the truth table, or from the Boolean expression corresponding to the sum function.

In this way, the hardware cost is also minimized.

The disadvantage of this solution is that a new design must be performed for each specific value of n (no modularity).

29

Modular combinational adder

More frequently, a modular approach is followed:

- **the 2 numbers X and Y are subdivided into bits (X_i, Y_i)**
- **pairs of bits X_i, Y_i are added, starting from the least significant bits**
- **the result at each stage must take into account the carry from the previous one.**

The modular approach is based on an elementary circuit called *full-adder.*

30

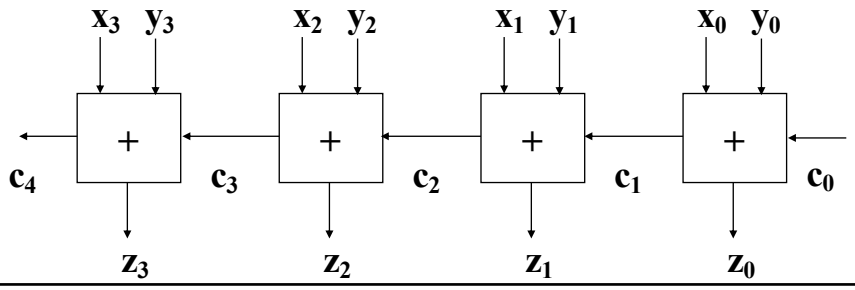
Ripple Carry Adder

It adds 2 numbers on *n* bits using combinational logic, only.

It is built by connecting *n* full-adder modules.

The time required for the generation of the last carry is equal to *nd*, where *d* is the delay of the single module.

The cost in hardware is proportional to *n*.



31

Carry-lookahaed adder

It is a further modular combinational solution, alternative to the ripple carry adder.

It allows to increase the speed in the generation of the result.

The Carry-Lookahaed Generator can generate the carry bit of each module based on the signals that come in parallel from all the modules.

These modules are a modified full-adder version in which two special signals are generated:

$$g_i = x_i y_i$$

$$p_i = x_i + y_i$$

32

Carry-Lookahaed Generator

The carry out signal of each module is given by

$$c_i=x_iy_i+x_ic_{i-1}+y_ic_{i-1}=g_i+p_ic_{i-1}$$

Similarly

$$c_{i-1}=g_{i-1}+p_{i-1}c_{i-2}$$

By substituting

$$c_i=g_i+p_ig_{i-1}+p_ip_{i-1}c_{i-2}$$

For a 4 bit carry-lookahaed adder we have:

$$c_0=g_0+p_0c_{in}$$

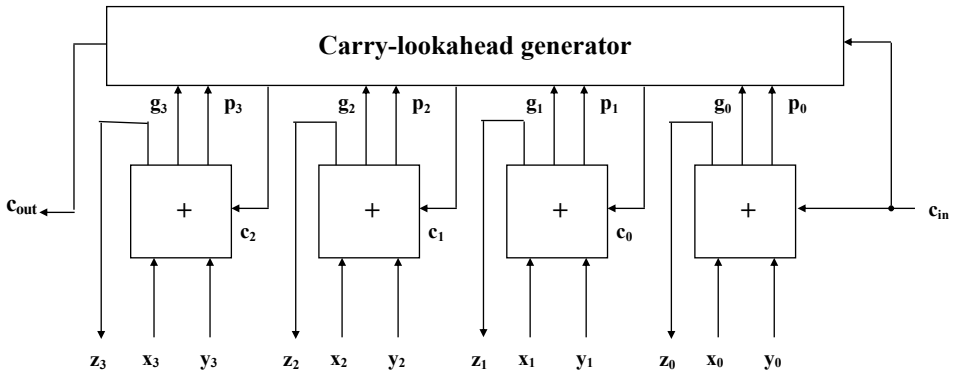
$$c_1=g_1+p_1g_0+p_1p_0c_{in}$$

$$c_2=g_2+p_2g_1+p_2p_1g_0+p_2p_1p_0c_{in}$$

$$c_3=g_3+p_3g_2+p_3p_2g_1+p_3p_2p_1g_0+p_3p_2p_1p_0c_{in}$$

33

Architecture



34

Advantages

If the delay of a 2-level circuit is called *d*, the delay of a carry-lookahead adder is equal to *3d*.

Disadvantages

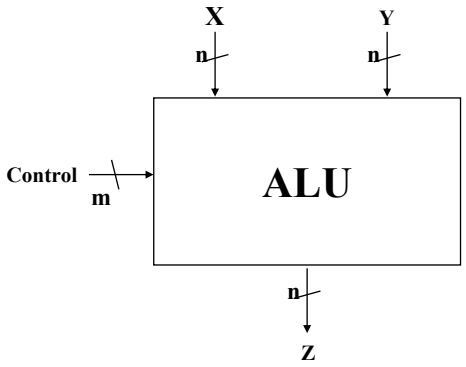
The cost in hardware terms grows exponentially with *n*.

For this reason, the method is generally inapplicable for high values of *n*.

35

ALU

***Arithmetic Logic Units* (ALUs) are combinational components that integrate in a single block the main arithmetic and logic functions (typically, addition, subtraction, negation, and, or, or, exor).**

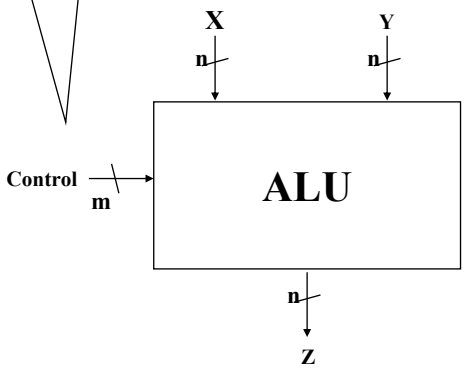


36

ALU

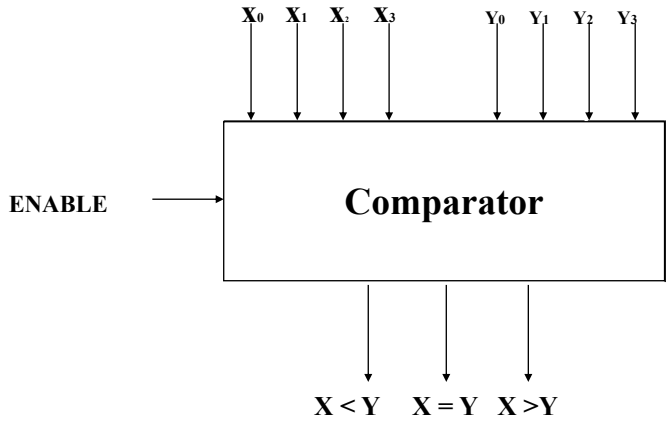
These signals determine the function to be performed by the ALU.

ic Units **(ALUs) are combinational hat integrate in a single block the main metic and logic functions (typically, additio ubtraction, negation, and, or, or, exor).**



37

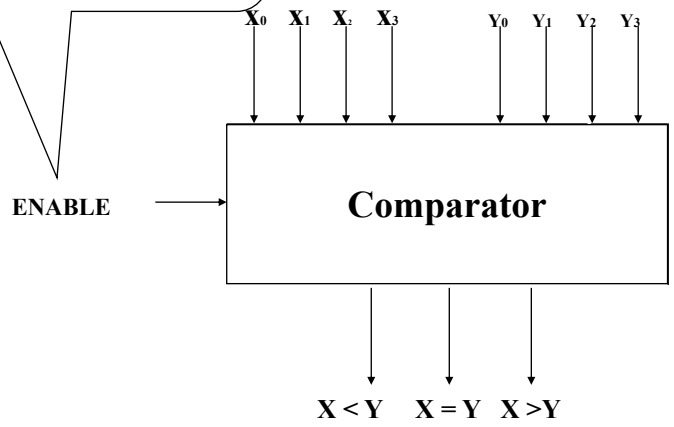
4-bit comparator



38

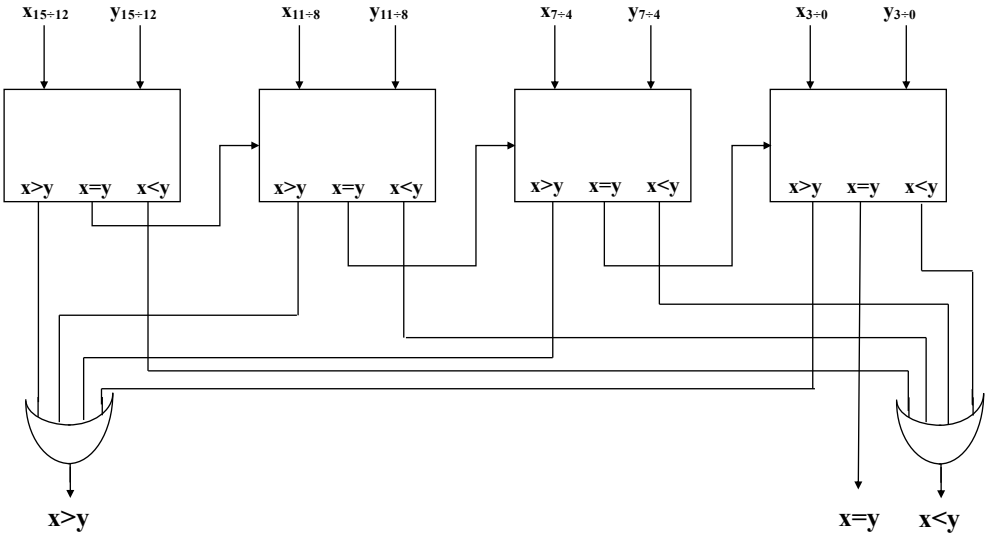
bit comparator

When the enable signal is inactive, no output is active (whatever the value of the other inputs).



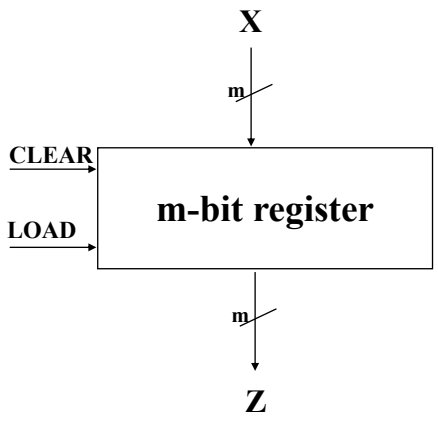
39

Connection of comparators



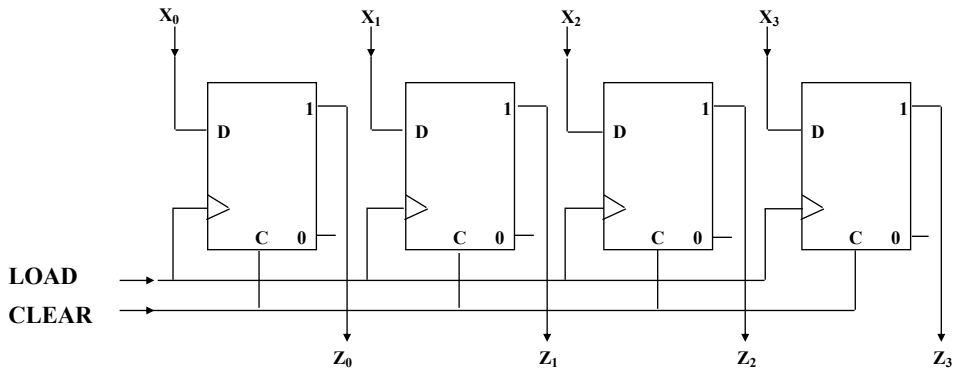
40

m-bit register



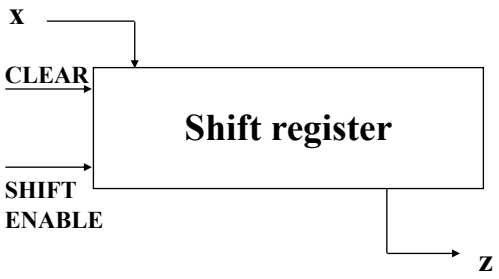
41

4-bit register (II)



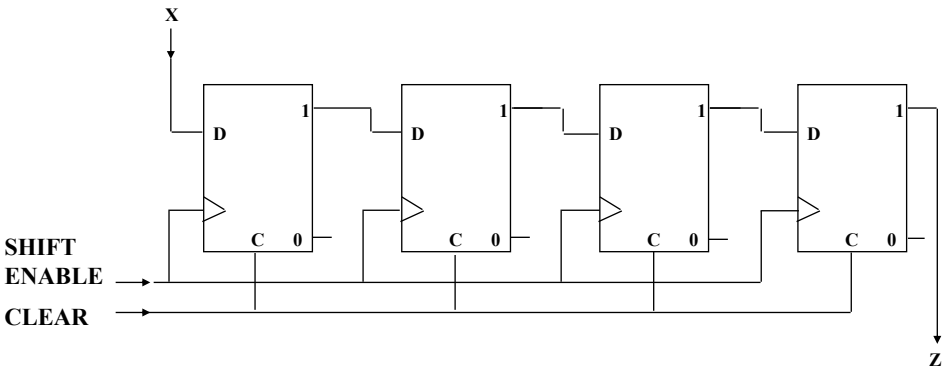
42

Shift Register



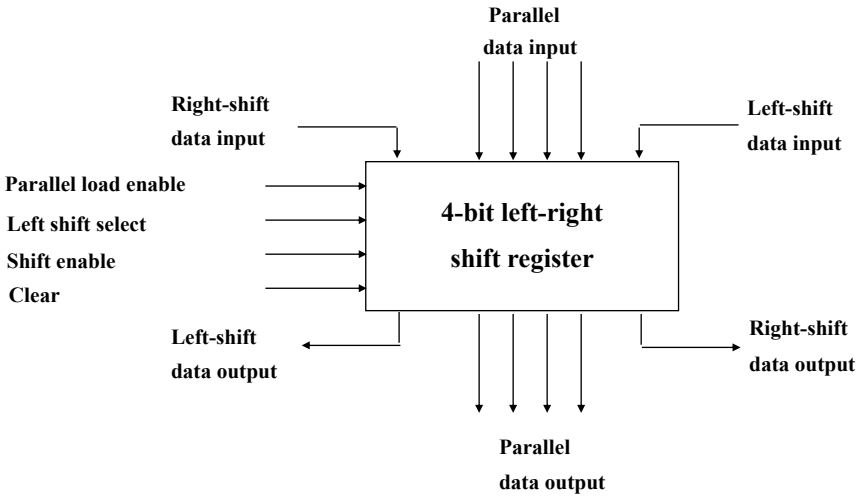
43

4-bit shift register: implementation



44

Universal shift register



45

Shift register usage

Examples

- **FIFO**
- **Serial-to-parallel and parallel-to-serial conversion**
- **Division / multiplication of unsigned integer numbers by power of 2.**

46

Counters

They evolve through k states in response to k impulses in input.

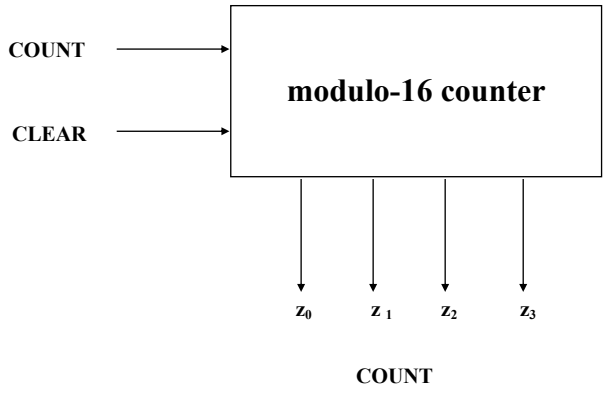
The state values allow you to count the number of pulses.

Types:

- ***up-down counter***
- ***programmable counter***
- ***binary/decimal/gray/BCD* counters.**

47

Simple counter



48

Usage of counters

Examples

- **as a *Program Counter* in a processor**
- **to generate clock signals (*frequency dividers*)**
- **event counters.**

49

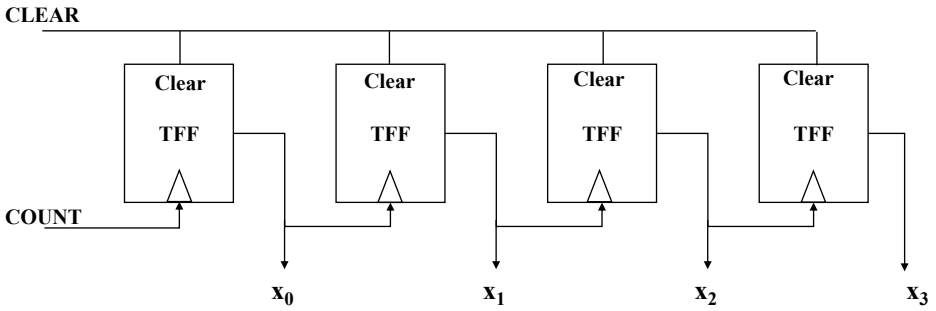
Counters: implementation

There are two main solutions:

- ***ripple counter* (or *asynchronous*)**
  - **The speed depends of the counter size**
- ***synchronous counter***
  - **All the outputs are updated at the same time.**

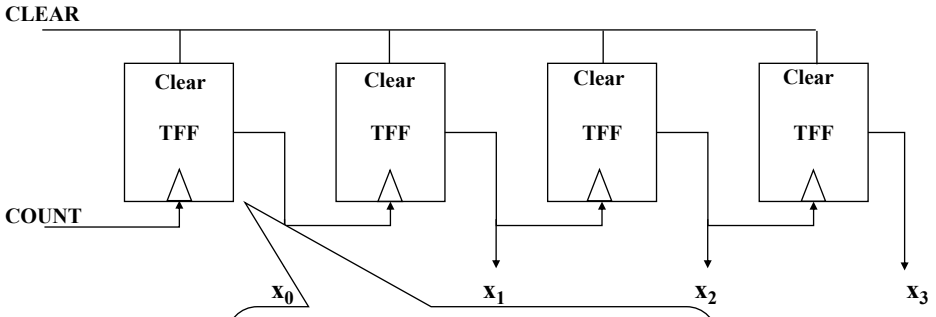
50

Asynchronous counter



51

Asynchronous counter

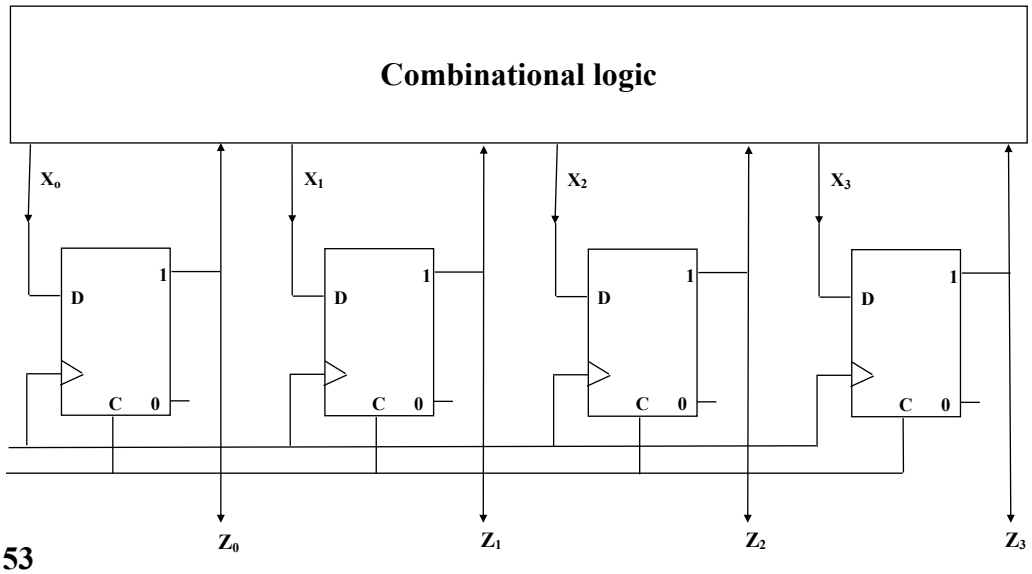


The T-type flip-flop is complemented at each clock pulse.

It can be obtained from D flip-flop by connecting the output to the input through an inverter.

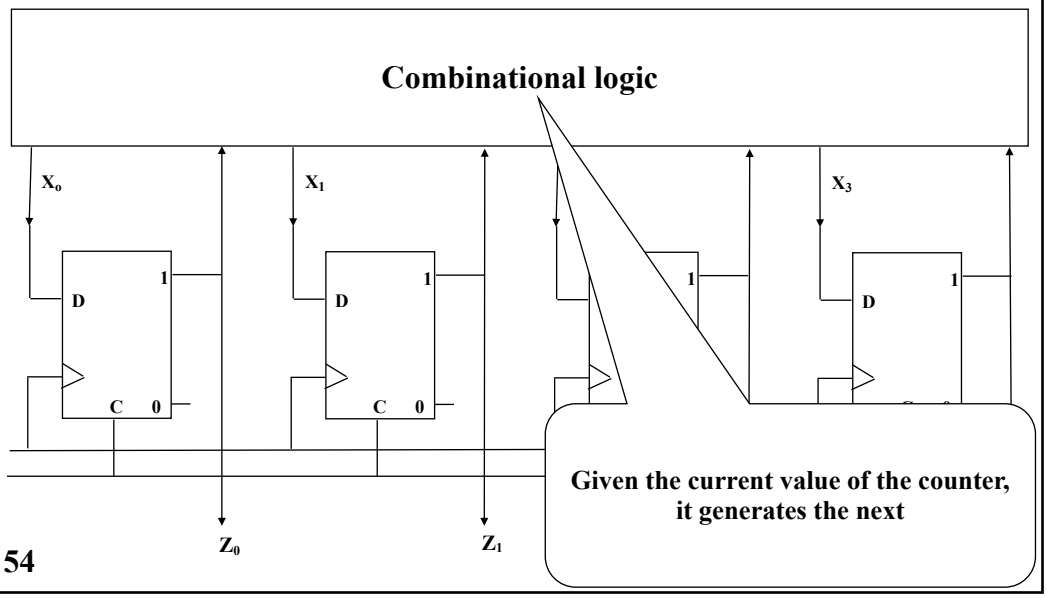
52

Synchronous counter



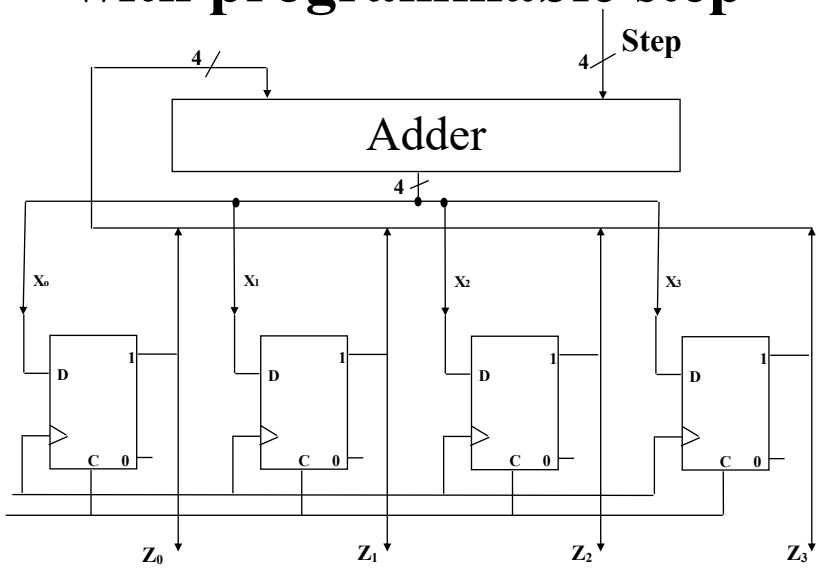
53

Synchronous counter



54

Modulo-16 synchronous counter with programmable step



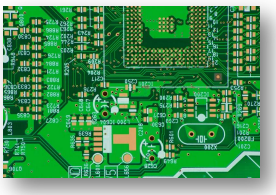
55

Bus

They allow data exchange between different components.

They can be realized

- within the single IC (Integrated Circuit)
- on a board (to connect different devices)
- as backplane interconnection.





56

Bus: cost

The cost of a bus can be high in terms of silicon area (if it connects remote modules on the same IC) and pins (if it is accessible from the outside of the chip).

Bus connection

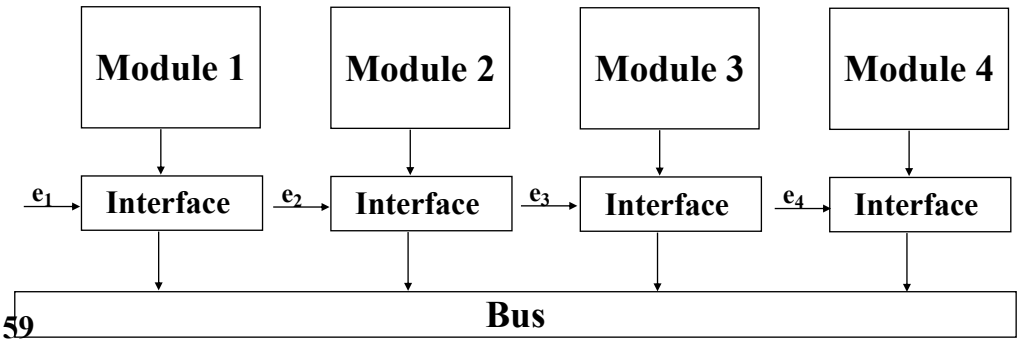
The generic device connected to a bus can

- **read the values on the bus**
- **write on the bus**
- **read or write according to the moments.**

Bus connection

At each time, one and only one of the devices connected to the bus can drive it.

The others have to assume a particular value known as Z (high impedance).



59

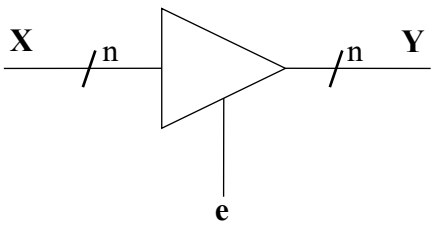
Tri-State Buffer

Buffers acting as interfaces to the bus are called *tri-state buffers*.

They have *n* data input signals X, 1 control input signal, and *n* data output signals Y.

The value of Y is

- **The one of X, if e = 1**
- **Z, if e = 0.**

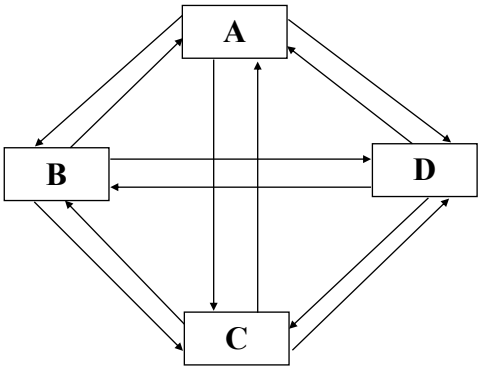


60

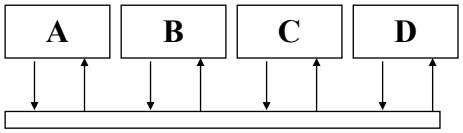
Bus types

Busses can be:

- *dedicated* (high cost, high performance)
- *shared* (low cost, low performance): they require a regulatory access mechanism.



Dedicated bus



Shared bus

61

Memories

They are composed of storage cells, each capable of storing a bit.

Cells are organized in words, each consisting of *n* bits.

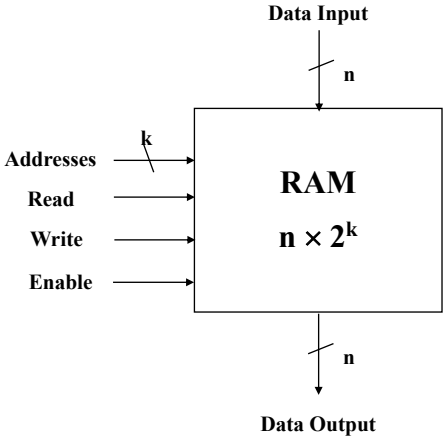
All the bits of a word are read and written together during the generic I / O operation to / from the memory.

Each word is characterized by an *address*, corresponding to an integer between 0 and m-1 (m being the number of words in the memory) encoded on k bit ($m = 2^k$).

Reading or writing a word requires applying the corresponding address to the memory inputs.

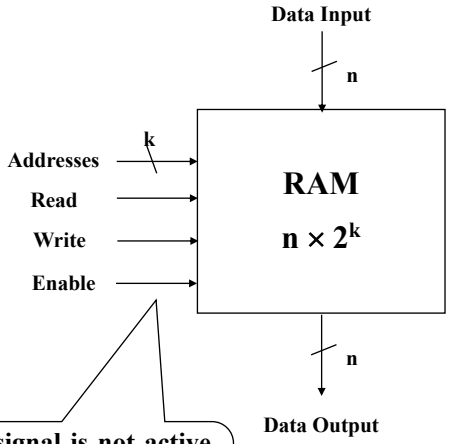
62

RAM



63

RAM



If the enable signal is not active, the memory is in standby mode (it keeps the content but does not react to the input signals).

6

RAM: behavior

Write cycle:

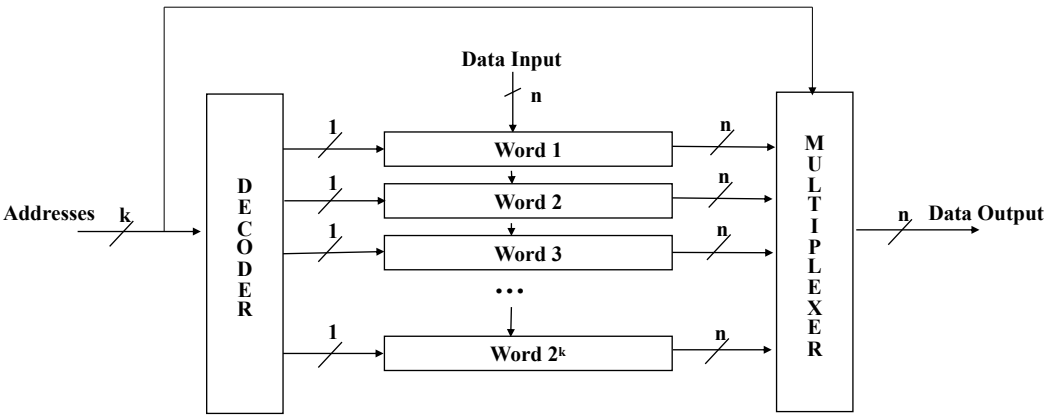
- **The address is provided**
- **The data is provided**
- **The *write* signal is activated.**

Read cycle:

- **The address is provided**
- **The *read* signal is activated**
- **The data can be read.**

65

RAM: internal structure



66

ROM

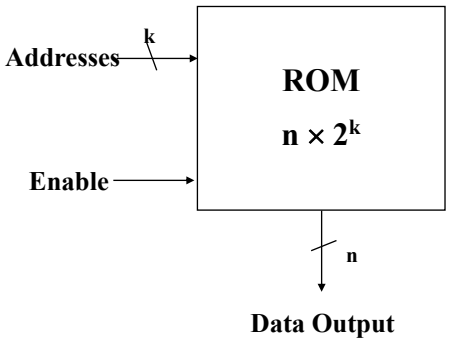
They are read-only memories.

They implement a combinational function.

The content is determined at the time of manufacturing, and can not be changed afterwards.

67

ROM



68

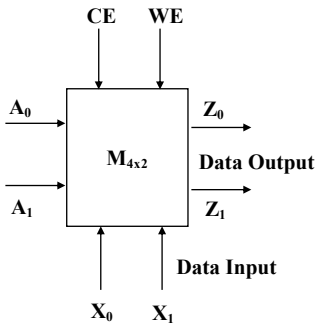
Memory bank

A memory bank is a set of properly interconnected memory modules that behave as a single memory (ROM, RAM or mixed) of a total size equal to the sum of the size of the component modules.

69

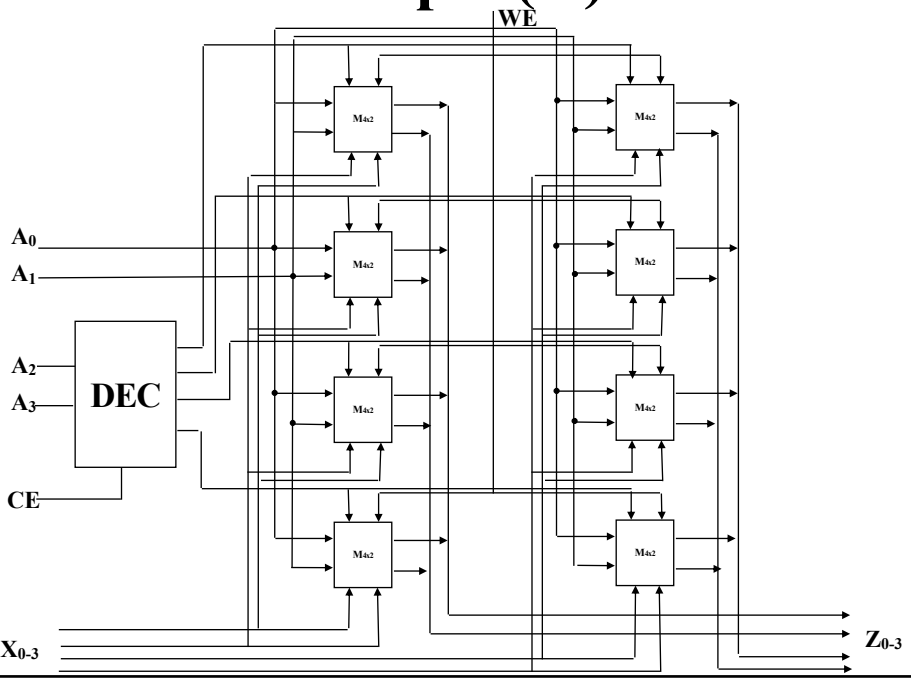
Example

We want to design a 16x4 bit RAM using the following module, containing 4 2-bit words each:



70

Example (II)



71

General rules

- All chips are connected in parallel to the data bus
- The address inputs of the various chips are driven in parallel from the least significant bits of the address
- The most significant address bits go to a decoder, which drives the enable (CE) signals of the various memory chips.

72